

Module 5

SPI — Protocol + RTL + Basic Testbench

Learning objectives

- SPI mode 0: SCLK idle low; capture on rising edge; change on falling edge; CS_N active low for fram...
- RTL block architecture: ``clk_div`` + ``spi_master`` hierarchy, FSM, and Mode 0 timing on the wire.
- Baseline verification: Directed stimulus, ``done``-based checks, and optional waveform review (8 SCLK...
- Path to Module 6: Same DUT, UVM agent with bus monitor on SCLK/MOSI/CS_N.

Prerequisites

- Modules 1–4 (spec→RTL, UVM, UART baseline, UART UVM).
- Verilator, Make, C++ compiler. No UVM required for spi_baseline.

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["SPI Protocol (Mode 0)"]

T2["Hands-on testbench (spi_base)"]

T1 --> T2

T2 --> N["SPI UVM+SV"]

Understand the SPI protocol (mode 0, signals), translate it to RTL (SPI master, clk_div), and verify with a **basic** (non-UVM) directed te...

Overview

- Module 5 is the second ****protocol**** module (after UART):

How to learn this module

Suggested learning path

- Read the detailed SPI learning guide:
SPI_LEARNING_GUIDE.md — what SPI is, what kind of protocol (s...
- Read the protocols + UVM overview:
LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 4.
When to Use Ba...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

CPP["sim_main.cpp\nclk, rst_n"]

DIV["clk_div\nclk_div_tick"]

SPI["spi_master\nMode 0 MSB first"]

CPP --> DIV

DIV --> SPI

Module 5: DUT / block hierarchy (see module5/examples/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

STIM["initial: start, data_in\n0x55, 0xAA"] --> M["spi_master"]

M --> WAIT["wait done"]

WAIT --> PASS["SPI baseline PASS"]

NOTE["Optional: VCD check\n8 SCLK edges, Mode 0"] -.-> PASS

Module 5: UVM layers, agents, and checkers for this phase.

1. Block hierarchy

- top_spi_baseline → `clk\_div` + `spi\_master`; C++ supplies `clk`/`rst\_n`.

2. SPI master RTL

- `clk_div`: ``clk_div_tick`` every ``DIVIDER`` clocks.
- `spi_master`: Mode 0; ``cs_n`` frames; MSB-first on ``mosi``; ``done`` when complete.
- Outputs: ``sclk`` (idle low), ``mosi``, ``cs_n`` — no ``miso`` in baseline.

3. Mode 0 timing on the wire

- Capture on rising SCLK; change MOSI on falling SCLK;
8 edges per byte.

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

STIM["start + data_in"] --> SPI["spi_master"]

SPI --> DONE["wait done"]

DONE --> PASS["baseline PASS"]

Module 5: how tests, checks, and metrics close verification goals.

1. Baseline goals

- Confirm Mode 0 serialization before SPI UVM (Module 6).

2. Directed test flow

- Pulse ``start`` + ``data_in`` (0x55, 0xAA);
 ``wait(done)``; read ``[PASS]`` messages.

3. Waveform learning (optional)

- Count 8 SCLK cycles per frame; verify MSB-first bit order on MOSI.
- Defer MISO, multi-slave CS, and random data to Module 6.

Syllabus topics

1. SPI Protocol (Mode 0)

- Signals: sclk (serial clock), mosi (master out, slave in), cs_n (active-low chip select). Optional:...
- Mode 0 (CPOL=0, CPHA=0): SCLK idles low; data captured on rising edge; data changed on falling edge.
- Timing: One bit per clk_div_tick; clk_div_tick derived from system clock via divider (e.g. DIVIDER=...

2. Hands-on testbench (spi_baseline)

- Directed test in ``top_spi_baseline.sv``: ``start``,
``data_in``, ``wait(done)``.
- Pass: ``[PASS]`` lines and ``SPI baseline test PASS``.

Hands-on examples

Module 5 self-check

```
$ ./scripts/module5.sh --help
```

Terminal: module5_check

```
[[0;36m===== [[0m
[[0;36mModule 5: Demo commands (SPI baseline) [[0m
[[0;36m===== [[0m
```

From repo root:

```
# 1) Environment sanity:
verilator --version
make --version
```

```
# 2) Run the SPI baseline example (basic TB, no UVM):
cd module5/examples/spi_baseline
make run
```

```
[[1;33m[INFO] [[0m See module5/EXAMPLES.md and module5/examples/spi_baseline/README.md for more.
```

Exercise scaffold

```
./scripts/module5.sh --scaffold
```


Example 1: SPI baseline

- ``dut/``: ``spi_master.v``, ``clk_div.v``
- ``top_spi_baseline.sv``: Top with `clk_div`, `spi_master`; directed stimulus and checks in initial blocks.
- ``sim_main.cpp``: C++ harness (clock, reset); simulation runs until ``$finish``.

Demo: SPI baseline

```
$ ./scripts/module5.sh --run
```

Terminal: spi_baseline

```
[0:36m===== [0m
[0:36mModule 5: Run spi_baseline example (logging to /mnt/d/proj/universal-verification-methodology
[0:36m===== [0m

[Thu May 28 15:27:15 CST 2026] Running make run
Compiling SPI baseline with Verilator...
verilator -sv --timing -Wall -Wno-fatal -Wno-TIMESCALEMOD -Wno-WIDTHTRUNC top_spi_baseline.sv dut/*.
/bin/sh: 1: verilator: not found
make: *** [Makefile:23: compile] Error 127
```

Practice & assessment

What you should know

- Describe SPI mode 0 and the role of sclk, mosi, cs_n, clk_div_tick.
- Explain the role of spi_master and clk_div.
- Run the spi_baseline example and interpret the directed test.
- Ready for Module 6: SPI UVM+SV verification.

Exercises

- Run spi_baseline
- Trace one transfer
- Mode 0 timing
- Optional: Waveforms

Assessment checklist

- Can describe SPI mode 0 (idle low, capture on rising, change on falling) and signals sclk, mosi, cs...
- Can explain the role of spi_master and clk_div in the RTL.
- Can run spi_baseline and interpret the directed test result.
- Ready for Module 6: SPI UVM+SV verification.

Summary & next steps

- Understand the SPI protocol (mode 0, signals), translate it to RTL (SPI master, clk_div), and verif...
- Complete module5/CHECKLIST.md
- Review module5/EXAMPLES.md and run each lab
- Next: SPI UVM+SV